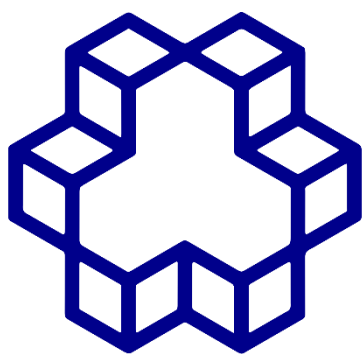




دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق - گروه مهندسی کنترل

مبانی سیستم‌های هوشمند

مینی پروژه اول

نام و نام خانوادگی	مهدی فیاض
شماره دانشجویی	۴۰۰۰۷۹۱۳
تاریخ	۱۴۰۳/۰۹/۰۸

۱ پرسش اول

۱.۱

درباره این مجموعه داده به صورت خلاصه توضیح دهید.

این دیتاست شامل اطلاعات حدود ۱۰,۰۰۰ مشتری کارت اعتباری است. هر مشتری با ویژگی‌هایی مثل سن، حقوق، وضعیت تاهل، سقف اعتبار کارت، نوع کارت اعتباری و ... مشخص شده است. حدود ۲۱ ویژگی برای هر مشتری وجود دارد.

هدف اصلی از این داده‌ست، پیش‌بینی مشتریانی است که احتمال ترک خدمات کارت اعتباری آن‌ها بالا است. این کار به بانک کمک می‌کند تا به صورت پیش‌گیرانه با این مشتریان ارتباط برقرار کند، خدمات بهتری ارائه دهد و تلاش کند تا تصمیم آن‌ها را تغییر دهد. یکی از چالش‌های اصلی در این مسئله، عدم تعادل داده‌ها است. تنها ۱۶.۰۷٪ از مشتریان در این داده‌ست، تصمیم به ترک خدمات داده‌اند. این عدم تعادل می‌تواند باعث شود مدل‌های پیش‌بینی به خوبی آموزش نبینند و کار چالشی بشه. همین دیگه

ویژگی‌های موجود در این مجموعه داده را نام ببرید.

ویژگی‌ها شامل :

1. پرچم ریزش - Attrition_Flag
2. شماره مشتری - CLIENTNUM
3. سن مشتری - Customer_Age
4. جنسیت - Gender
5. تعداد افراد تحت تکفل - Dependent_count
6. سطح تحصیلات - Education_Level
7. وضعیت تاهل - Marital_Status
8. دسته درآمدی - Income_Category
9. دسته کارت اعتباری - Card_Category
10. تعداد ماه‌هایی که مشتری با بانک بوده است - Months_on_book
11. تعداد کل روابط با بانک - Total_Relationship_Count
12. تعداد ماه‌های غیرفعال در ۱۲ ماه گذشته - Months_Inactive_12_mon
13. تعداد تماس‌های مشتری با بانک در ۱۲ ماه گذشته - Contacts_Count_12_mon
14. سقف اعتبار کارت اعتباری - Credit_Limit

15. مانده حساب جاری - Total_Revolving_Bal
 16. متوسط موجودی قابل استفاده - Avg_Open_To_Buy
 17. تغییر کل مبلغ هزینه شده در Q4 ماه های نسبت به Q1 - Total_Amt_Chng_Q4_Q1
 18. کل مبلغ هزینه شده - Total_Trans_Amt
 19. کل تعداد تراکنش ها - Total_Trans_Ct
 20. تغییر کل تعداد تراکنش ها در Q4 نسبت به Q1 - Total_Ct_Chng_Q4_Q1
 21. متوسط نسبت استفاده از اعتبار - Avg_Utilization_Ratio
- منظور از Q1 ماه های ربع اول سال است

چه تعداد نمونه در این مجموعه داده موجود است؟

نمونه ۱۰۱۲۷

۲.۱

درون فایل کد آورده شده است.

۳.۱

درون فایل کد آورده شده است. برای تبدیل ویژگی های کیفی (categorical) به داده های عددی (کمی) از دو روش استفاده کردیم:

- 1- به ازای هر دسته بندی در هر ویژگی یک ستون ایجاد کرده و سطرهایی که در آن دسته قرار گرفته بودند ۱ و بقیه صفر می شوند. مثلاً برای تبدیل ویژگی شهر تولد به داده ی عددی به صورت زیر عمل می کنیم:

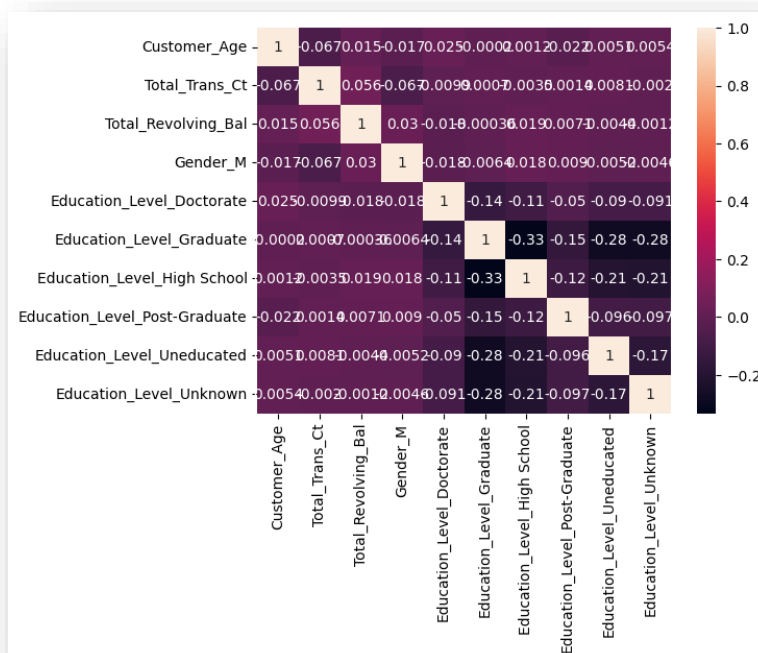
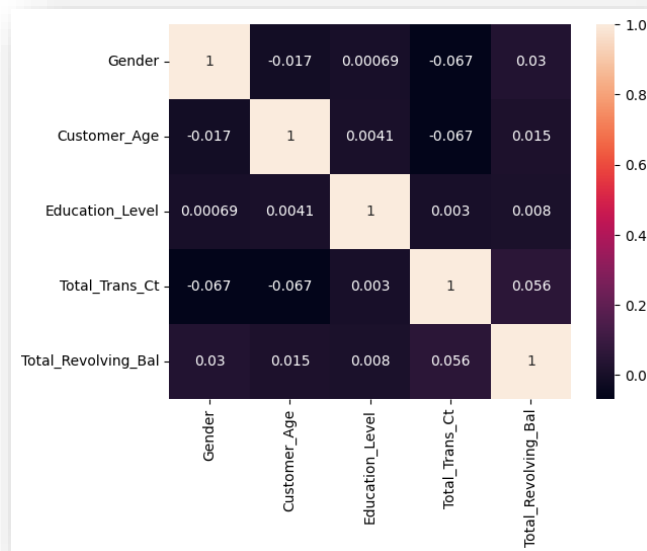
```
Original DataFrame:
  city  age
0  Tehran  25
1  Shiraz  30
2  Isfahan  35
3  Tehran  28

Encoded DataFrame:
  age  city_Isfahan  city_Shiraz  city_Tehran
0  25             0             0             1
1  30             0             1             0
2  35             1             0             0
3  28             0             0             1
```

- 2- می توانیم به هر دسته ای یک عدد طبیعی را بنگاریم (map کنیم 😊). اگر ویژگی کمی مان ترتیب داشت این کار را به صورت هدفمند و در غیر اینصورت به صورت دلخواه انجام می دهیم. مثلاً برای همان مثال شهر:

	city	city_encoded
0	Tehran	2
1	Shiraz	1
2	Isfahan	0
3	Tehran	2

در این قسمت سوال، در بخش کد نیز ما ویژگی های سطح تحصیلات و جنسیت را برای رسم heatmap انتخاب کردیم و با هر دو روش همبستگی ها را رسم کردیم که در زیر نقشه ها را مشاهده می کنید.



۴.۱

درون فایل کد آورده شده است.

داده‌ی Nan در میان دیتاها نداشتیم.

۵.۱

ویژگی Attrition Flag دارای چند کلاس است؟

دو کلاس. کلاس مشتریان موجود و مشتریان از دست‌رفته 😞

نام کلاس‌های موجود در این ویژگی چیست؟

کلاس‌های Existing Customer و Attrited Customer

پخش داده موجود در این ویژگی را به صورت یک pie plot نمایش دهید.

درون فایل کد آورده شده است.

ویژگی Attrition Flag که می‌خواهیم مدلی برای پیش‌بینی آن بسازیم، دارای عدم تعادل است. تحقیق

کنید که آیا این عدم تعادل در عملکرد مدل نهایی تأثیر دارد یا نه. توضیح دهید.

عدم تعادل داده‌ها (Class Imbalance) در مسئله‌های یادگیری ماشین، به ویژه در مدل‌هایی که برای پیش‌بینی ویژگی‌هایی مانند استفاده می‌شوند، می‌تواند تأثیر قابل توجهی بر عملکرد مدل داشته باشد. این مسئله زمانی رخ می‌دهد که یکی از کلاس‌ها به طور قابل توجهی بیشتر از دیگری باشد (مثلاً تعداد داده‌های مربوط به کلاس "فعال" بسیار بیشتر از "غیرفعال" باشد) (مانند مسئله‌ی stock در حفر چاه‌های عمیق که در کلاس درس مطرح شد)

تأثیر عدم تعادل داده بر عملکرد مدل:

مدل‌های یادگیری ماشین تمایل دارند که کلاس غالب را به دلیل فراوانی بیشتر، بهتر شناسایی کنند. این امر ممکن است باعث شود مدل کلاس اقلیت را نادیده بگیرد یا در پیش‌بینی آن دچار خطا شود.

به قول استاد مدل چیزی را یاد می‌گیرد که به آن نشان دهی. وقتی یک کلاس را بیشتر نشان بدهی، آن را بیشتر یاد می‌گیرد.

معیارهایی مانند Accuracy ممکن است در مسائل نامتعادل گمراه‌کننده باشند. حتی اگر مدل تمام نمونه‌ها را به کلاس غالب نسبت دهد، دقت ممکن است بالا به نظر برسد، اما مدل عملاً ناکارآمد است. (اگر ۹۵ درصد داده‌ها مربوط به کلاس یک باشد، یک چوب هم می‌تواند کارایی ۹۵ درصد به ما بدهد 😊)

مدلهایی که با داده‌های نامتعادل آموزش می‌بینند، معمولاً در شناسایی الگوهای مرتبط با کلاس اقلیت ضعف دارند، که باعث کاهش قابلیت پیش‌بینی و تعمیم‌پذیری آن‌ها در داده‌های جدید می‌شود.

		Predicted		
		Positive	Negative	
Actual	Positive	True Positive	False Negative	Recall/Sensitivity $\frac{TP}{TP + FN}$
	Negative	False Positive	True Negative	Specificity $\frac{TN}{TN + FP}$
		Precision $\frac{TP}{TP + FP}$	Negative Predictive Value $\frac{TN}{TN + FN}$	Accuracy $\frac{TP + TN}{TP + FN + TN + FP}$

چه راهکارهایی برای اصلاح این مشکل وجود دارد؟ تحقیق کنید.

برای کاهش اثرات منفی این عدم تعادل، روش‌های زیر قابل استفاده هستند:

1. توازن داده‌ها (Data Balancing):

- Oversampling: افزایش تعداد نمونه‌های کلاس اقلیت با استفاده از روش‌هایی مانند SMOTE یا تکرار داده‌ها.
- Undersampling: کاهش تعداد نمونه‌های کلاس غالب.
- ترکیب دو روش فوق.

2. استفاده از مدل‌های مقاوم به عدم تعادل: الگوریتم‌هایی مانند Random Forest یا XGBoost می‌توانند وزن

بیشتری به کلاس اقلیت بدهند. تنظیم وزن نمونه‌ها (Class Weights) در الگوریتم‌هایی مانند Logistic Regression.

3. انتخاب معیارهای ارزیابی مناسب: معیارهایی مانند F1-Score، Precision، Recall و ROC-AUC که حساسیت به هر دو کلاس را نشان می‌دهند، می‌توانند بینش بهتری از عملکرد مدل ارائه دهند.

4. افزودن داده‌های مصنوعی یا شبیه‌سازی شده: داده‌های اضافی مرتبط با کلاس اقلیت را می‌توان از طریق روش‌های شبیه‌سازی تولید کرد.

اگر بخواهیم از یک الگوریتم برای متعادل کردن مجموعه داده استفاده کنیم، باید این کار را قبل از تقسیم بندی داده به بخش های آموزش و آزمون انجام دهیم یا پس از آن؟ توضیح دهید.

اگر متعادل سازی قبل از تقسیم بندی انجام شود، ممکن است داده های مصنوعی یا باز نمونه گیری شده (مانند داده های حاصل از Oversampling یا Undersampling) به طور مشترک در هر دو مجموعه آموزش و آزمون ظاهر شوند. این اتفاق باعث نشت اطلاعات بین دو مجموعه و در نتیجه ارزیابی نادرست عملکرد مدل می شود.

متعادل کردن داده ها باید بعد از تقسیم بندی داده ها انجام شود و تنها روی مجموعه آموزش اعمال شود.

یکی از اهداف بخش آزمون این است که عملکرد مدل در داده های واقعی با توزیع اصلی ارزیابی شود. اگر متعادل سازی بر روی کل داده ها (پیش از تقسیم بندی) انجام شود، این توزیع اصلی از بین می رود و نتایج آزمون دیگر نشان دهنده عملکرد واقعی مدل نخواهند بود. در واقع ما می توانیم برای آموزش دادن به مدل مان از داده های ساختگی استفاده کنیم، اما در بخش تست که می خواهیم عملکرد حقیقی مدل مان را بسنجیم باید با داده ها واقعی کار کنیم که دقت به دست آمده معتبر باشد.

۶.۱

بدون متعادل کردن داده ها، مدل خود را آموزش دهید.

داخل فایل کد آورده شده است.

یک الگوریتم متعادل سازی مجموعه داده را معرفی کرده و پس از متعادل کردن داده، مدل خود را آموزش دهید

SMOTE (Synthetic Minority Over-sampling Technique) یک تکنیک oversampling است که برای متعادل کردن داده های نامتعادل در مسائل طبقه بندی استفاده می شود. در داده های نامتعادل، یک کلاس (کلاس اکثریت) تعداد نمونه های بیشتری نسبت به کلاس دیگر (کلاس اقلیت) دارد. این عدم تعادل می تواند باعث شود که مدل های یادگیری ماشین به خوبی آموزش نبینند و در پیش بینی کلاس اقلیت دچار مشکل شوند.

روش کار SMOTE: این کار با استفاده از مراحل زیر انجام می شود:

انتخاب یک نمونه از کلاس اقلیت به صورت تصادفی.

پیدا کردن k نزدیک ترین همسایه برای این نمونه در کلاس اقلیت (با استفاده از معیار فاصله مانند فاصله اقلیدسی).

انتخاب یکی از این k همسایه به صورت تصادفی.

ایجاد یک نمونه مصنوعی جدید در فضای ویژگی، بین نمونه اصلی و همسایه انتخاب شده.

این فرایند برای تعداد مشخصی از نمونه‌های مصنوعی جدید تکرار می‌شود تا داده‌ها متعادل شوند.

فرمول ایجاد نمونه مصنوعی:

$$x_{new} = x_i + \text{rand}(0, 1) * (x_{zi} - x_i)$$

که در آن:

x_i نمونه اصلی از کلاس اقلیت

x_{zi} همسایه انتخاب شده به صورت تصادفی از بین k نزدیک‌ترین همسایه

مزایای SMOTE: ۱- با افزایش تعداد نمونه‌های کلاس اقلیت، مدل می‌تواند الگوهای مربوط به این کلاس را بهتر یاد

بگیرد و در نتیجه، عملکرد آن در پیش‌بینی این کلاس بهبود می‌یابد. ۲- کاهش **overfitting**: با ایجاد نمونه‌های

مصنوعی جدید، تنوع داده‌ها را افزایش می‌دهد و از **overfitting** جلوگیری می‌کند.

معایب SMOTE: ۱- ممکن است نمونه‌های مصنوعی ایجاد کند که در فضای ویژگی واقعی نیستند و ۲- افزایش تعداد

نمونه‌ها، زمان آموزش مدل افزایش می‌یابد.

SMOTE در بسیاری از مسائل طبقه‌بندی با داده‌های نامتعادل مانند تشخیص تقلب، تشخیص بیماری و ... استفاده

می‌شود.

۲ پرسش دوم

۱.۲

درون فایل کد آورده شده است.

۲.۲

سه معیار برای سنجش عملکرد مدل‌های رگرسیون معرفی کنید و هر یک را توضیح دهید.

1. میانگین مربع خطا (MSE): میانگین مربعات تفاوت‌ها بین مقادیر واقعی و پیش‌بینی شده است. این معیار

حساسیت زیادی به خطاهای بزرگ دارد.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2. میانگین قدر مطلق خطا (MAE): نشان‌دهنده میانگین اختلاف‌های مطلق است و نسبت به MSE کمتر تحت تأثیر خطاهای بزرگ قرار می‌گیرد.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

3. R-squared: (ضریب تعیین) این معیار مشخص می‌کند که چه بخشی از تغییرات در متغیر وابسته (خروجی) توسط مدل توضیح داده می‌شود. این معیار بین ۰ و ۱ است، که ۱ نشان‌دهنده برازش کامل مدل با داده‌ها است.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

SS_{res} : مجموع مربعات باقیمانده‌ها (اختلاف بین مقادیر پیش‌بینی شده و مقادیر واقعی)

SS_{tot} : مجموع مربعات کل (اختلاف بین مقادیر واقعی و میانگین آنها)

۳.۲

یک مدل رگرسیون خطی ۱۷ درجه اول (بدون استفاده از توابع آماده) روی داده مورد نظر آموزش دهید. به نظر شما آیا یک مدل خطی درجه اول می‌تواند به خوبی داده مورد نظر را تخمین بزند؟ توضیح دهید. درون فایل کد آورده شده است.

خیر. زیرا ذات داده‌ها سهمی شکل و درجه دو است و لذا مدل خطی درجه یک برای آن مناسب نیست و قطعاً خطای زیادی را به همراه خواهد داشت.

۴.۲

توضیح دهید با افزایش داده آموزش چه اتفاقی برای خطاهای آزمون و آموزش می‌افتد.

همانطور که در نمودار نیز مشاهده می‌شود خطای آموزش به تدریج زیاد شده و به مقدار واقعی اش نزدیک می‌شود. وقتی در ابتدا یک داده را به مدل نشان می‌دهیم آن داده را به راحتی حفظ می‌کند و خط را از آن می‌گذرانند و لذا خطا صفر

است. با زیاد کردن داده های آموزش گذراندن خطی که از تمام آن ها بگذرد سخت تر شده و خطا افزایش می یابد. اما برای دیتا های تست که در واقع میزان واقعی صحت عملکرد مدل ما است، با افزایش نمونه های آموزش، این خطا کاهش می یابد. چرا که مدل ما نمونه های بیشتری را دیده و با مدل اصلی مواجه می شود. اما به صورت کلی چون ذات دیتا ها یک تابع سهمی درجه ۲ است و مدل ما ضعیف است (undefiting) خطاهای رسم شده روال نرمال ندارند 😊.



۵.۲

برای انجام فعالیتی، خطای انسان برابر ۱۱ است. یک مدل یادگیری ماشین برای انجام همین فعالیت آموزش داده شده است که خطا آموزش آن برابر ۱۰ است. اگر برای آموزش این مدل از داده بیشتری استفاده کنیم، آیا می توانیم خطا مدل را به اندازه خطا انسان کاهش دهیم؟ توضیح دهید.

به صورت کلی بستگی به نوع مسئله ی ما دارد. اما اگر این خطای زیاد ناشی از ضعیف و ساده بودن مدل ما نسبت به مدل حقیقی باشد، با افزودن داده نمی توان این کمبود را جبران کرد. به صورت کلی خطای تست از خطای آموزش بیشتر است و لذا در این مسئله به صورت پیش فرض خطای حقیقی از ۱۰ نیز بیشتر است.

اگر مدل ما **underModel** باشد با افزودن داده نمی توان خطا را کاهش چشمگیر داد اما اگر این خطا ناشی از کمبود داده باشد و ساختار مدل ما پتانسیل بالقوه تخمین آن را داشته باشد، در آن صورت خطا به دلیل کمبود داده بوده و با افزودن داده می توان خطا را کاهش داد.

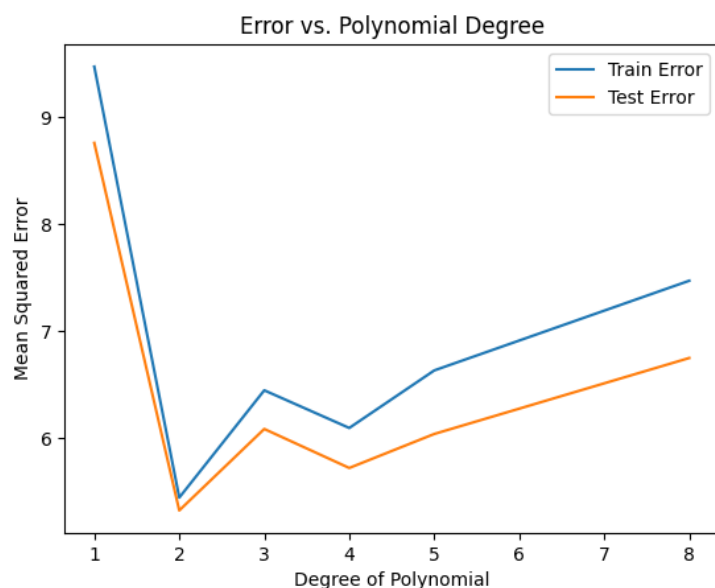
۶.۲

درون فایل کد آورده شده است.

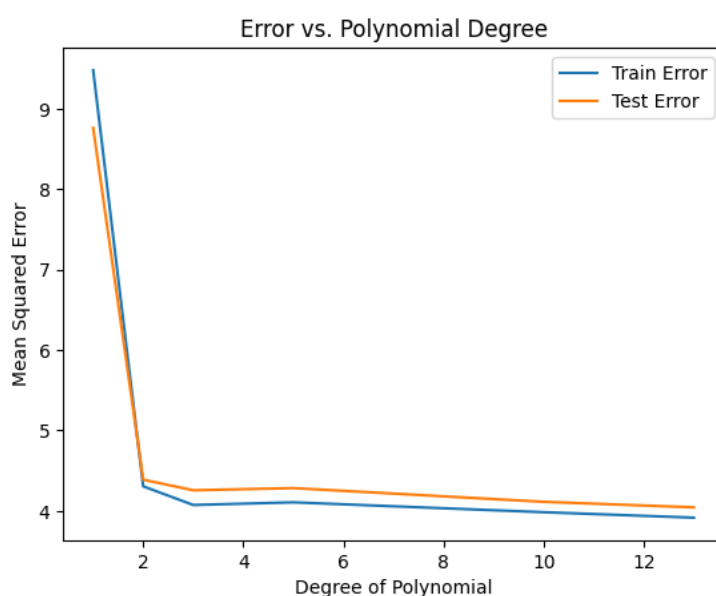
آیا با افزایش تعداد جمله های مدل، خطای آزمون همواره کاهش میابد؟ توضیح دهید.

خیر. همانطور که در شکل نیز پیداست، با افزودن

جملات از یک جا به بعد (در این مدل، درجه ۲) خطا سیر صعودی می گیرد. دلیل آن نیز، **overfitting** شدن مدل است. بهترین درجه در این مدل، همانطور که به صورت شهودی انتظار میرفت نیز درجه ۲ است. با افزایش درجه، مدل سعی میکند خودش را با داده های آموزش منطبق کند و حتی نویزها را نیز یاد می گیرد اما با آمدن داده ی جدید دیگر جوابگوی آنها نیست. در واقع خطای واریانس مدل بالا می رود.



البته اگر فقط از $x^{**degree}$ استفاده نکنیم و به صورت متوالی از x^1 تا x^n از ویژگی ها استفاده شود، نمودار به صورت زیر می شود. در این حالت نیز، احتمال **overmodel** شدن وجود دارد و خطای تست با افزودن جملات بالا می رود.



البته سوال کمی گنگ بود. اینکه هر بار یک درجه به جملات قبل اضافه کنیم؟؟ یا اینکه هر بار فقط خود X باشد و ما یک درجه ی دلخواه به آن اضافه کنیم؟؟ یا اینکه هر بار از درجه یک تا n را یعنی n ستون به ویژگی ها اضافه کنیم؟؟ شکل ها متفاوت می شد. اما همه جوره تست شد. نتیجه کلی اینکه درجات زیاد هم خوب نیست. و باعث **overfitting** میشه. تمام.

۷.۲

Linear Regression

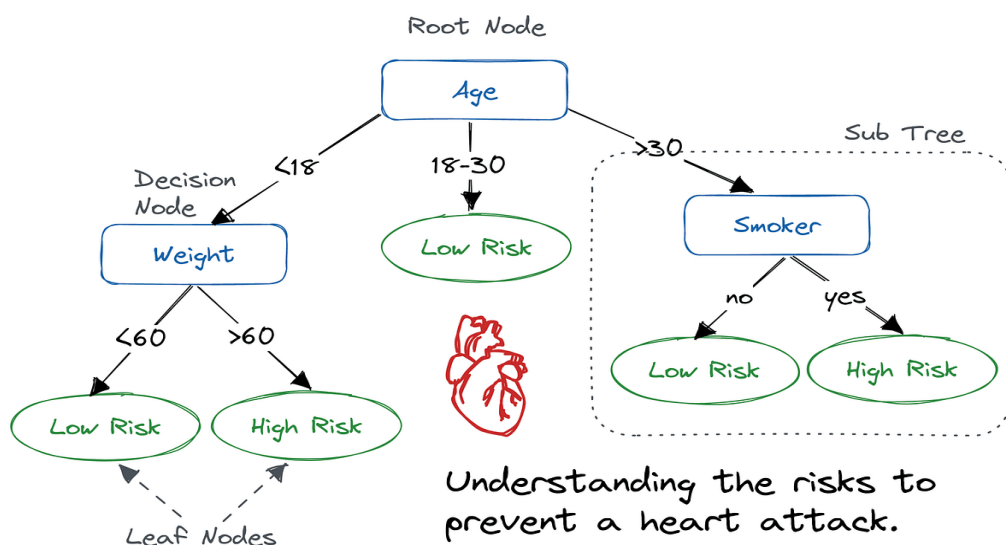
با این مدل زیادی آشنا شدیم. ترکیب خطی از ورودی ها می سازد و سعی می کند به روش هایی به خصوص گرادینان نزولی این ضرایب را تصحیح کند تا تابع هزینه به کمترین مقدار خودش برسد.

$$y = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$$

این مدل بسیار ساده، تفسیرپذیر و برای مسائل پیش بینی کارایی خوبی دارد اما به داده های پرت حساس است و برای مدل های غیرخطی جواب گو نیست

Decision Tree Regression

رگرسیون درخت تصمیم (Decision Tree Regression) یکی از الگوریتم های پرکاربرد در یادگیری ماشین است. درخت تصمیم رگرسیونی، همانطور که از نامش پیداست، به شکل یک درخت است. هر گره داخلی در این درخت نشان دهنده یک ویژگی (feature) از داده ها است و هر شاخه نشان دهنده یک مقدار ممکن برای آن ویژگی است. برگ های درخت نیز مقادیر پیش بینی شده را نشان می دهند.



مراحل ساخت یک درخت تصمیم رگرسیونی به طور خلاصه عبارتند از:

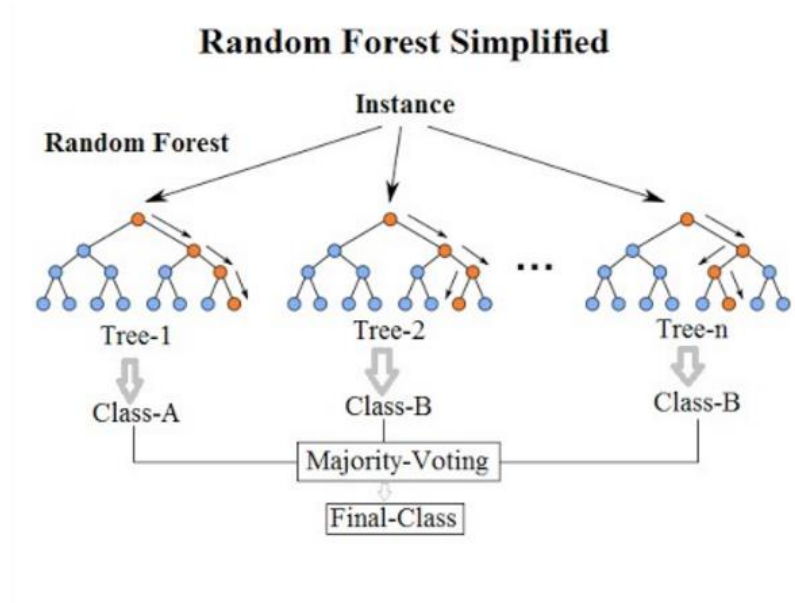
- انتخاب ویژگی ریشه: ابتدا بهترین ویژگی برای تقسیم داده‌ها انتخاب می‌شود. بهترین ویژگی به ویژگی‌ای گفته می‌شود که بیشترین توانایی را در کاهش ناهمگونی داده‌ها داشته باشد.
- ایجاد شاخه‌ها: داده‌ها بر اساس مقدار ویژگی انتخاب شده در گره ریشه به چندین زیرمجموعه تقسیم می‌شوند.
- تکرار فرآیند: برای هر زیرمجموعه جدید، مراحل ۱ و ۲ تکرار می‌شود تا زمانی که یک شرط توقف مانند حداکثر عمق درخت یا حداقل تعداد نمونه در هر برگ برآورده شود.
- پیش‌بینی: برای پیش‌بینی مقدار یک نمونه جدید، آن نمونه از ریشه درخت شروع کرده و با توجه به مقادیر ویژگی‌های آن، در درخت پیمایش می‌شود تا به یک برگ برسد. مقدار پیش‌بینی شده برای آن نمونه، میانگین مقادیر هدف در آن برگ خواهد بود.

از خوبی‌های درخت تصمیم این است که تفسیرپذیری بالایی دارد و شهودی طور هست، همینطور داده‌ها در این روش، می‌توانند نامنظم و از انواع مختلف باشند و همینطور نیازی به نرمال سازی و ... ندارند. اما در مقادیر پیوسته و یا اگر ویژگی‌ها زیاد باشند درخت تصمیم روش مناسبی نیست و همینطور می‌توانند به شدت تحت تأثیر نویز در داده‌ها قرار بگیرند و منجر به ایجاد درختانی پیچیده و بیش از حد برازش (همون **overfit** خودمون) شده شوند.

Random Forest

جنگل تصادفی (Random Forest) یکی از پرکاربردترین و قدرتمندترین الگوریتم‌های یادگیری ماشین است که در مسائل طبقه‌بندی و رگرسیون به کار می‌رود. چگونه کار می‌کند؟

- 1- ساخت چندین درخت تصمیم: جنگل تصادفی با ساختن تعداد زیادی درخت تصمیم کار می‌کند. هر درخت تصمیم بر روی یک زیرمجموعه تصادفی از داده‌های آموزشی و یک زیرمجموعه تصادفی از ویژگی‌ها آموزش داده می‌شود.
- 2- ترکیب نتایج: پس از ساخت درختان تصمیم، برای هر نمونه جدید، هر یک از درختان یک پیش‌بینی انجام می‌دهند. پیش‌بینی نهایی جنگل تصادفی با رای‌گیری اکثریت (در طبقه‌بندی) یا میانگین‌گیری (در رگرسیون) از نتایج تمام درختان تعیین می‌شود.



تنظیم پارامترهای جنگل تصادفی:

- تعداد درختان: تعداد درختان تصمیم در جنگل
- عمق درختان: حداکثر عمق هر درخت تصمیم
- تعداد ویژگی‌ها در هر تقسیم: تعداد ویژگی‌هایی که برای هر تقسیم در درخت تصمیم در نظر گرفته می‌شود

مزایای جنگل تصادفی:

- دقت بالا: با ترکیب چندین درخت تصمیم، جنگل تصادفی اغلب دقت بالاتری نسبت به یک درخت تصمیم منفرد دارد.
- کاهش بیش‌برازش: انتخاب تصادفی زیرمجموعه داده‌ها و ویژگی‌ها به کاهش بیش‌برازش کمک می‌کند.
- قابلیت مدیریت داده‌های نامنظم: جنگل تصادفی می‌تواند با انواع مختلف داده‌ها، از جمله داده‌های عددی، گسسته و حتی داده‌های از دست رفته، کار کند.
- اهمیت ویژگی‌ها: جنگل تصادفی می‌تواند اهمیت هر ویژگی در مدل را مشخص کند.

C	B	A
جنگل تصادفی	درخت تصمیم	ویژگی
پیچیده‌تر	ساده	پیچیدگی
پایین‌تر	بالا	تفسیرپذیری
بالاتر	پایین‌تر (در حالت کلی)	دقت
کمتر	بیشتر	بیش‌برازش